

Lab 5 Report 110550022

Introduction

This report explores the implementation and evaluation of a DQN agent implemented for Lab 5, with focus on enhancements inspired by the Rainbow DQN framework.

Implementation

DQN Bellman error

$$L_{\text{DQN}}(\theta) := \frac{1}{2} \sum_{(s,a,r,s') \in D} \left(r + \gamma \max_{a' \in \mathcal{A}} Q(s', a'; \bar{\theta}) - Q(s, a; \theta) \right)^2,$$

```
with torch.no_grad():  
    target_q_values = rewards + self.gamma * (1 - terminateds) * self.target_net(next_states).max(1)[0]
```

Q values were already provided, so we'll need to work out the first 2 terms, i.e. the target Q values obtained from the Bellman equation. r is the stored immediate reward value, and γ is the discount factor constant. The equation passes next states into the target Q network and finds the max value across each action ($\max(\text{dim}=1)$). Only the first component is kept since $\text{torch.max}()$ returns a tuple of values and indices. The entire target q value calculation is wrapped within a `torch.no_grad()` clause to prevent back propagation from updating the target net.

One important distinction here from the original code base is that the target Q value is set to zero only on termination rather than both termination and truncation, as hinted by the transition composition (`dones` instead of `truncateds`). When a run is truncated, the Q values should still be maintained rather than being set to zero. This change is primarily effective at stabilizing later training episodes for the cartpole problem, since the maximum frame count is set to 500 in the gymnasium environment by default, when max episode steps =

500, the last frame is always truncated.

```
td_errors = q_values - target_q_values
#
squared_errors = td_errors ** 2
```

```
if self.use_replay_buffer:
    loss = torch.mean(torch.tensor(weights, device=self.device) * squared_errors)
else:
    loss = torch.mean(squared_errors)
```

After the calculation of target Q values, the loss is set as the mean squared error between the two values, and the 1/2 factor is omitted for simplicity.

DDQN

The only difference between DQN and DDQN lies in the derivation of target Q values. DDQN uses the Q values when taking the action predicted by the Q net.

```
with torch.no_grad():
    next_actions = self.q_net(next_states).argmax(1, keepdim=True)
    # when it is terminated, q_value should be zero. But q_value should still be calculated when truncated
    target_q_values = \
        rewards + self.gamma * (1 - terminateds) * self.target_net(next_states).gather(1, next_actions).squeeze(1)
```

Next actions are obtained by finding the index with the highest q value within each batch, it is then used as a filter to select the q values these actions correspond to.

PER Memory Buffer

```
def __init__(self, capacity, alpha, beta_start, beta_annealing_steps):
    self.capacity = capacity
    self.alpha = alpha
    self.beta = beta_start
    self.beta_increment_per_step = (1.0 - beta_start) / beta_annealing_steps

    self.buffer = []
    self.priorities = np.zeros((capacity,), dtype=np.float32)
    self.pos = 0
```

Other than the values already in the template, I've added an additional `beta_increment_step` variable to perform linear annealing on beta. This approach is mentioned in both the PER and Rainbow DQN papers.

```
def __len__(self):
    return len(self.buffer)
```

A length attribute is added to the class so that `len(memory)` works for lists and PER buffer.

```
def add(self, transition):
    ##### YOUR CODE HERE (for Task 3) #####
    max_prio = np.max(self.priorities[:len(self.buffer)]) if self.buffer else 1.0

    if len(self.buffer) < self.capacity:
        self.buffer.append(transition)
    else:
        self.buffer[self.pos] = transition

    self.priorities[self.pos] = max_prio
    self.pos = (self.pos + 1) % self.capacity
    ##### END OF YOUR CODE (for Task 3) #####
```

In line 6 of the PER pseudo code from the original paper, a transition is stored with maximum priority within the current buffer, or 1 if the memory is empty.

Initialize replay memory $\mathcal{H} = \emptyset$, $\Delta = 0$, $p_1 = 1$

6: Store transition $(S_{t-1}, A_{t-1}, R_t, \gamma_t, S_t)$ in $\mathcal{H}$ with maximal priority $p_t = \max_{i < t} p_i$

The position of the latest index is stored in `self.pos`, and taking it modulo capacity will make the newest entry overwrite the oldest.

```
def sample(self, batch_size):
    ##### YOUR CODE HERE (for Task 3) #####
    N = len(self.buffer)
    prios = self.priorities[:N]

    probs = prios / prios.sum()

    indices = np.random.choice(N, batch_size, p=probs)
    samples = [self.buffer[idx] for idx in indices]

    weights = (N * probs[indices]) ** (-self.beta)
    weights /= weights.max()

    return samples, indices, weights
```

When sampling, the priorities are normalized into probabilities of an index being sampled. The weights are then calculated using the formula in the slides, and normalized again by the maximum weight.

The indices are returned for use when updating priorities.

```
def update_priorities(self, indices, errors):
    ##### YOUR CODE HERE (for Task 3) #####
    for idx, error in zip(indices, errors):
        self.priorities[idx] = (error + 1e-6) ** self.alpha
    ##### END OF YOUR CODE (for Task 3) #####
```

The td error between q and target q values are passed into this function along with indices returned from the sample function to update their priorities. A small value is added to the error so that even if error is zero it still has a probability to be resampled and trained on.

```
def beta_step(self):
    self.beta = min(1.0, self.beta + self.beta_increment_per_step)
```

Beta step is called each training step to anneal beta.

There are several other changes that were introduced to the DQN agent class as well.

```
if self.use_replay_buffer:
    self.memory.add((state, action, reward, next_state, terminated)) #
else:
    self.memory.append((state, action, reward, next_state, terminated))
    if len(self.memory) > self.memory_capacity:
        self.memory.pop(0)
```

Add is called instead of append

```
if self.use_replay_buffer:
    mini_batch, indices, weights = self.memory.sample(self.batch_size)
else:
    mini_batch = random.sample(self.memory, self.batch_size)
```

Memory.sample instead of random.sample

```
if self.use_replay_buffer:
    loss = torch.mean(torch.tensor(weights, device=self.device) * squared_errors)
else:
    loss = torch.mean(squared_errors)
```

The squared errors are multiplied by the weights.

```
errors = td_errors.abs().detach().cpu().numpy()

if self.use_replay_buffer:
    self.memory.update_priorities(indices, errors=errors)
    self.memory.beta_step()
```

Pass errors into update priorities and anneal beta

Multi-step Return

```
class MultiStepPrioritizedReplayBuffer(PrioritizedReplayBuffer):
```

The multi-step version of the buffer is implemented as an inherited class of the single-step version.

```
def __init__(self, capacity, n_steps, gamma, alpha, beta_start, beta_annealing_steps):
    super().__init__(capacity, alpha, beta_start, beta_annealing_steps)
    self.n_steps = n_steps
    self.gamma = gamma
    self.history = deque(maxlen=n_steps)
```

A few additional variables are initialized, including the number of steps, the gamma constant, and a history buffer of length n_steps.

```
def add(self, transition):
    self.history.append(transition)

    if len(self.history) < self.n_steps:
        return

    #states, actions, rewards, next_states, terminated
    n_step_return = 0
    next_state, terminated = self.history[-1][3], self.history[-1][4]

    for k in range(self.n_steps):
        n_step_return += self.gamma ** k * self.history[k][2] # gamma^k * rt+k

        if self.history[k][4]: # if terminated, not truncated. If its truncated, the buffer will get cleared in the next step
            next_state = self.history[k][3]
            terminated = True
            break

    transition = self.history[0][0], self.history[0][1], n_step_return, next_state, terminated # use n step transition instead

    # same as super.add()
    super().add(transition)
```

Only the add function needs to be changed. At each call, the transition is added into the history buffer. When history is full, the states and actions of the oldest entry in the buffer are taken as the states and actions of the packed transition. The n_step return value is calculated by iterating through the history buffer and accumulating the weighted rewards using the formula in the slides. When encountering a terminal state, we take that state as the next state of the packed transition, otherwise the newest entry in the buffer is used.

Finally, the packed transition is added into the buffer by calling the add function of single step PER.

```
if isinstance(self.memory, MultiStepPrioritizedReplayBuffer):  
    self.memory.history.clear() # clear history to prevent cross-env returns
```

At the start of each episode, the history buffer has to be cleared to prevent transitions across different environments.

Weights & Biases

Wandb allowed me to track the reward trend to check for any instabilities or if it is plateauing on a non-optimal solution. Graphing the results also takes less effort.

CNN Architecture

I followed rainbow DQN and chose to use the dueling network proposed in <https://arxiv.org/abs/1511.06581>.

Gradient Clipping

The Duel CNN paper suggests to clip the gradients using norm = 10.

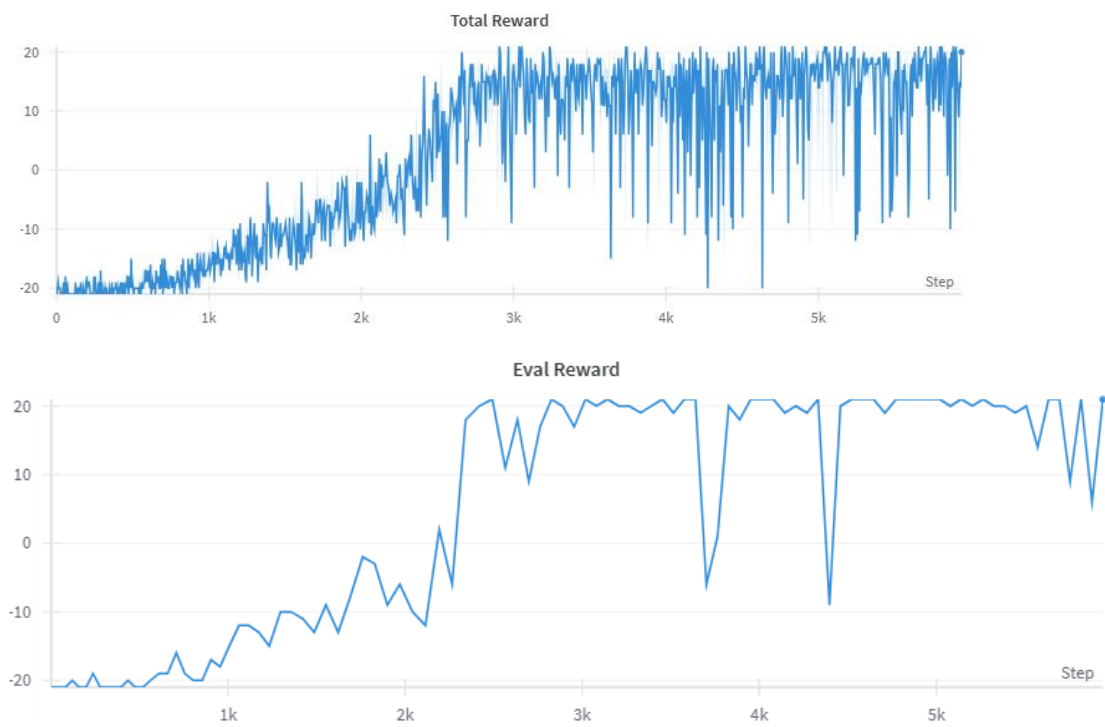
Analysis and discussions

Task 1:

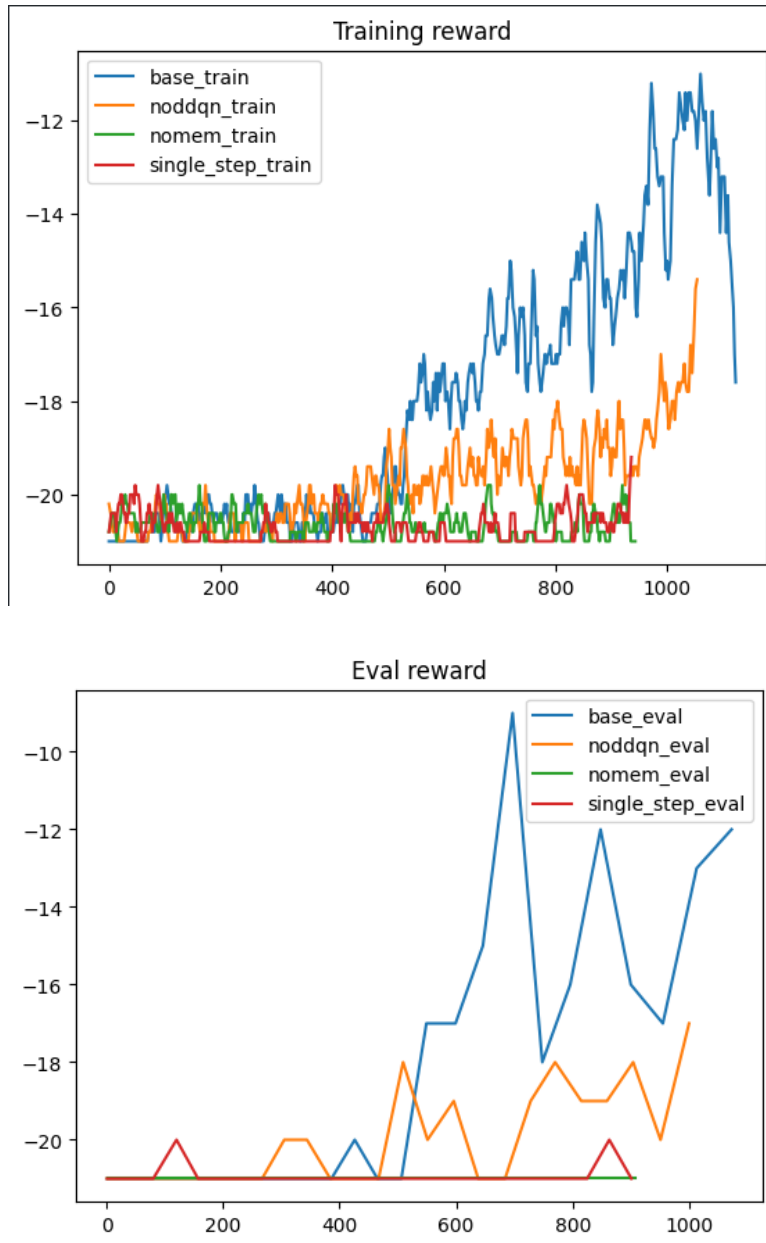




Task 2:



Task 3



For task 3, I've only trained the model for 500 episodes, so it doesn't take too long. From the graphs it's pretty evident that the base settings (ddqn + multistep PER) have the best performance. We can also tell that multi-step gives the biggest boost by comparing base and single step. DDQN is slightly more ineffective than multistep, but still gives a sizable efficiency increase, while single step PER barely gives any improvement (single step vs no mem, its effect is still noticeable from the minor bumps on the graph).

Additional analysis of other training

strategies

Unused Actions: In Pong-v5, there are 6 available actions by default. However, half of the actions are not used, namely FIRE, LEFT FIRE, and RIGHT FIRE as the ball is always served by the opponent. This issue can be circumvented by wrapping the environments within an ActionWrapper, though that would require change in the testing script and even though it did speed up the training process, I wasn't able to tune it to improve the scores of tasks 3 before due (still over 1 mill steps needed) so it was ultimately dropped.

Soft updating, Huber Loss: These didn't affect the training too much.